



More data beats clever algorithms, but better data beats more data.

Peter Norvig (born 1956)

American Computer Scientist

18.1 Overview of the Historical Developments

Even though research activities were strongly reduced during “first AI winter”, which lasted from the early 1970s until 1980, there still was some progress. One of the important developments was the introduction of nonlinear step functions (such as the sigmoidal function). Furthermore, in 1974, PAUL JOHN WERBOS described a new type of training process where the errors were *backpropagated* through the layers of the network¹ [10]. While WERBOS was the first to describe backpropagation, this approach was also (re)discovered in other research communities. It took until approximately 1985 when DAVID E. RUMMELART, GEOFFREY E. HINTON, and RONALD J. WILLIAMS introduced the algorithm for backpropagation of errors in the context of *multilayer perceptrons* (MLPs) in [8]. This made many of the developments possible, which lead to today’s deep learning.

The following sections start by introducing the most important building blocks for *deep learning* (DL): new types of activation functions and the backpropagation algorithm.

¹ The PhD thesis [10] is the original publication; additionally, the thesis was later published in the book [11].

18.2 Activation Functions

In the previous chapter, we only encountered one type of activation function, the binary step function (or Heaviside function). However, it turns out that for back-propagation, non-zero gradients are required. Also, having a certain degree of nonlinearity in the activation function generally helps with the training and increases the predictive power of a network. In fact, the activation function is the first place where a *non-linearity* can enter the network. Thus, there are many options for mathematically formulating activation functions. One of their common properties is that they all are elementary mathematical functions. They are always monotonic functions, i.e., their function values are always increasing. Additionally, some of them have the property to saturate at large absolute values: they are *squashing functions* that map an arbitrary value range to a bounded output. Figure 18.1 shows an overview of some activation functions and the respective mathematical formulations; further discussions can be found, e.g., in [12] and in the references therein.

Activation Functions



Usually, all hidden layers use the same activation function. The output layer (as we already saw in the previous chapter) may use a different type of activation function. This depends on the kind of prediction (classification, regression) that is to be made. For example, for regression problems, the last layer is a layer with linear activation.

18.2.1 Step Function

The *step function* is the simplest approach, which, however, does not work for multi-class classification problems because it can only give two different outputs, 0 and 1. Additional, the gradient is zero, which poses a problem for back-propagation.

18.2.2 Linear Function

Using a *linear activation function*, which is also called *identity function*, the performance of the neural network will not or only slowly improve due to the constant gradient during the back-propagation: a constant gradient implies that it is insensitive to the value of the input. If all hidden layers have linear activation functions, then each layer is a linear function of the previous one and, effectively, all layers can be replaced by a single layer with linear activation functions.

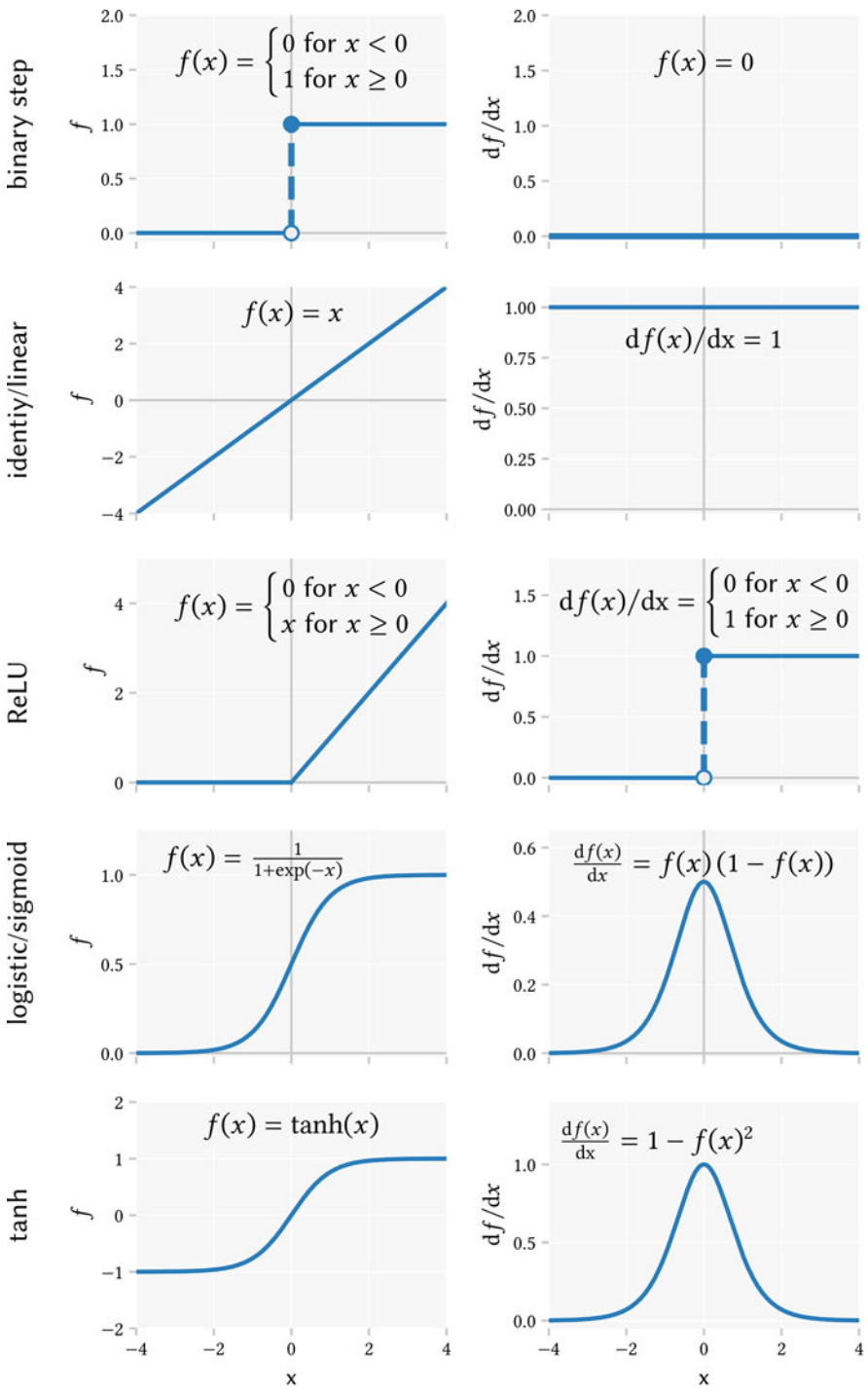


Fig. 18.1 Activation functions (left column) and their derivatives (right column)

18.2.3 Sigmoidal Function

The *logistic* or *sigmoid* activation function is a nonlinear function. The function is non-symmetric with $f(x) > 0$ around $x = 0$. As a consequence, the outputs of all neurons have the same sign, which makes training less stable. Due to its range of the function values of $[0, 1]$, it is used in the context of predicting probabilities. The function has only small gradients further away from the center; the fact that $df(x)/dx = f(x) \cdot (1 - f(x))$ implies that for every iteration the values are getting smaller (below we will discuss this in the context of the “vanishing gradient problem”).

18.2.4 The Family of ReLU Functions

One of the often used type of activation functions today is the *rectified linear unit* (ReLU) function: The ReLU activation function looks extremely simple: it consists of a horizontal line and an 45° -inclined line. It is not differentiable at $x = 0$, which might lead to problems during gradient-based optimization. A property of the ReLU function is that it is scale-invariant. However, one of the side effects of this mathematical formulation is that it does not activate all neurons at the same time; due to the zero gradient, it even might lead to “dead neurons” that *never* get activated. This is also the reason, why we typically prefer to initialize the neurons directly connected to ReLU activation functions with a slightly positive bias by, e.g., adding a small “offset” to the initial random values.

A variant of ReLU that aims to remedy the problem of dead neurons is the *leaky ReLU*. This variant avoids non-zero function values and derivatives by adding a small slope to the region of $x \leq 0$. The slope is the “leakage” parameter:

$$f(x) = \begin{cases} x & \text{if } x > 0, \\ 0.01x & \text{otherwise.} \end{cases} \quad (18.1)$$

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1 & \text{if } x > 0, \\ 0.01 & \text{otherwise.} \end{cases} \quad (18.2)$$

The first equation can also be implemented as $f(x) = \max(0.01x, x)$.

Furthermore, the *parameterized ReLU* treats the leakage parameter as a parameter that needs to be identified during the training. These are just a few examples, there are many more formulations, some of which are specialized to particular network architectures.

18.2.5 Hyperbolic Tangent Function

The second, most commonly used type of activation function, is the hyperbolic tangent (tanh) function. It is very similar to the sigmoid function, but it is symmetric around 0. It also has a steeper gradient, which can be seen by the value of the derivative at $x = 0$ in Fig. 18.1. Usually, it performs better than the sigmoid function. As opposed to ReLU, it is not scale-invariant.

18.3 Back-Propagation: Introduction and Example

Already a few years after ROSENBLATT published the *perceptron*, [6], he also published the concept of a *multilayer perceptron* (MLP), which contained an input layer, a hidden layer, and an output layer [7]. The hidden layer was initialized with random values for the weights. However, there was no mechanism such that the hidden layer was able to adjust the weights. Because a DL network has the ability to adjust the weights in the hidden layer, the multilayer perceptron would not yet be considered a proper DL network.

Even though the gradient descent method in the context of training was indirectly established through the delta rule, at this point the concepts of a cost function in conjunction with gradient descent minimization did not exist for *artificial neural networks* (ANNs). The great achievement of RUMMELHARD et al. [8] was to generalize the gradient descent framework to the context of training multilayer neural networks with nonlinear activation functions in a way such that *all* weights get automatically adjusted.

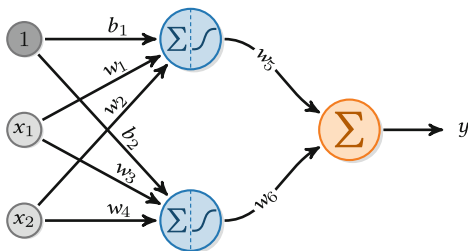
18.3.1 The General Concept

The *back-propagation* algorithm (which is sometimes also called in short “back-prop” algorithm) consists of two processes: the *forward pass* (or propagation phase) where the inputs are propagated through the network and the output predictions are computed, and the *backward pass* during which the gradient of the loss function of the output layer is computed *with respect to all weights*; this gradient is then recursively propagated through the network from the output layer all the way to the input layer. The chain rule for differentiation is key for finding how each individual weight of all hidden layer(s) has to be adjusted (the so-called *weight update*) in order to minimize the loss function.

18.3.2 A First Example

We will now investigate a very simple system, similar to that from Sect. 17.4.1, and introduce the backpropagation formalism. The network has two input nodes, two

Fig. 18.2 Network with one hidden layer and a sigmoidal activation function. The output is just aggregated, and there is no activation function



nodes in the hidden layer, and a single output. The hidden nodes contain sigmoidal activation functions, and the whole network is shown in Fig. 18.2.

Furthermore, we only consider a single data record consisting of two features and a single target variable, $\mathcal{P} = (x_1, x_2; y)$.

Activation Function

The activation functions of the hidden layer both have the following mathematical form:

$$\varphi(z) = \frac{1}{1 + \exp(-z)} \quad (18.3)$$

Loss and Cost Function

Additionally, we need to quantify the “badness” of the performance of the network. This is done using a quadratic *loss function*, in analogy to what has been introduced in Sect. 12.3.3 and in particular in Definition 12.6. For each training example, it is

$$\mathcal{L}(\mathbf{w}; (\mathbf{x}, y)) = \frac{1}{2} \left(y - y^{\text{pred}}(\mathbf{w}; x_1, x_2) \right)^2. \quad (18.4)$$

Note that the factor of $1/2$ is there to simplify the equation during computing the derivative; this does not have any effect on finding the minimum of the cost function below. From this equation, we see that $\mathcal{L}$ depends on the current values of all weights that are varied during the training, as well as on a given record from the training dataset.

The loss function gives a single value for each data record. The *cost function* on the other hand is the average loss for all records of the dataset. Similar to Definition 12.5 we can now define the cost function $\mathcal{J}$. Writing this directly for only a single data record hides some details; thus, we write it first in all generality as the average cost of M data records:

$$\mathcal{J} = \frac{1}{2M} \sum_{i=1}^M \left(y_i - y^{\text{pred}}_i(\mathbf{w}; x_{i1}, x_{i2}) \right)^2, \quad (18.5)$$

which, for a dataset that consists of only one record, would simplify to

$$\mathcal{J} = \frac{1}{2} \left(y - y^{\text{pred}}(\mathbf{w}; x_1, x_2) \right)^2 . \quad (18.6)$$

The first and second features for several data records are given by the column vectors X_1 and X_2 (compare the definition of the feature matrix, Sect. 3.3.3), the vector of all labels is Y . The resulting cost function is then a function that depends on six weights and two bias values as variables:

$$\mathcal{J} = \mathcal{J}(w_1, \dots, w_6, b_1, b_2; X_1, X_2; Y) . \quad (18.7)$$

Words of advice



In the field of deep learning, the difference between cost and loss is very often not recognized, and ML practitioners only talk about the “loss functions.” To be consistent with the formulations from earlier chapters, we will continue to differentiate between cost and loss.

Forward Pass

The feed-forward step consists of evaluating the network prediction y^{pred} for a given input, in analogy to Sect. 17.4.1, together with computing the cost. The quantities can be best computed starting from the output layer and then moving toward the input layer. The cost function can then be written as

$$\mathcal{J} = \frac{1}{2} (y - a_3)^2 , \quad (18.8)$$

where we replaced y^{pred} with the result from the agglomeration function a_3

$$a_3 = b_3 + y_1 w_5 + y_2 w_6 = y^{\text{pred}} , \quad (18.9)$$

This function performs the weighted summation, and y_1 and y_2 are the outputs of the two units of the hidden layer. They can be computed as

$$y_1 = \varphi(a_1) \quad (18.10)$$

$$y_2 = \varphi(a_2) . \quad (18.11)$$

where φ is the sigmoidal activation function. The weighted summations are:

$$a_1 = b_1 + x_1 w_1 + x_2 w_2 \quad (18.12)$$

$$a_2 = b_2 + x_1 w_3 + x_2 w_4 . \quad (18.13)$$

For brevity, we refrained from stating the function arguments. For example, we automatically implied the short notation $a_1 \equiv a_1(w_1, w_2, b_1; x_1, x_2)$.

Backward Pass

Training implies to find the weights that minimize the cost function,

$$\text{Find } w_1, \dots, w_6, b_1, b_2 \text{ such that } \mathcal{J} \rightarrow \text{MIN} . \quad (18.14)$$

Thus, to solve this minimization problem with the *gradient descent* (GD) method, we will need to compute the derivatives $\partial \mathcal{J} / \partial w_1 \dots \partial \mathcal{J} / \partial w_6$, as well as $\partial \mathcal{J} / \partial b_1$ and $\partial \mathcal{J} / \partial b_2$. Again, we start from the output of the network and then move through the network toward the input layer.

We start to compute the derivative w.r.t. one of the weights of the output node using the *chain rule* and Eq. 18.8:

$$\frac{\partial \mathcal{J}}{\partial w_5} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial w_5} = -(y - a_3) \cdot y_1 \quad (18.15)$$

Hint for Writing the Chained Products of Derivatives



Start writing the chain of derivatives from the right to the left. For example, if the partial derivative is w.r.t. w_1 then search for the term that contains w_1 : it is a_1 . Thus the rightmost term is $\partial a_1 / \partial w_1$. Next, watch out for the equation that contains a_1 : it is y_1 , giving you the second derivative term $\partial y_1 / \partial a_1$. Then, search for the equation that contains y_1 : it is a_3 , and we found a third term. It is $\partial a_3 / \partial y_1$. Last but not least, the function that contains a_3 is $\mathcal{J}$, hence, $\partial \mathcal{J} / \partial a_3$ and we're done.

The result is:

$$\frac{\partial \mathcal{J}}{\partial w_1} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial y_1} \cdot \frac{\partial y_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1}$$

The derivation of $\partial \mathcal{J} / \partial w_6$ follows in analogy:

$$\frac{\partial \mathcal{J}}{\partial w_6} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial w_6} = -(y - a_3) \cdot y_2 . \quad (18.16)$$

Next, the expressions for $\partial \mathcal{J} / \partial b_2$, $\partial \mathcal{J} / \partial w_3$, and $\partial \mathcal{J} / \partial w_4$ are derived:

$$\frac{\partial \mathcal{J}}{\partial b_2} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial y_2} \cdot \frac{\partial y_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial b_2} = -(y - a_3) \cdot w_6 \cdot y_2(1 - y_2) \quad (18.17)$$

$$\frac{\partial \mathcal{J}}{\partial w_3} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial y_2} \cdot \frac{\partial y_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_3} = -(y - a_3) \cdot w_6 \cdot y_2(1 - y_2) \cdot x_1 \quad (18.18)$$

$$\frac{\partial \mathcal{J}}{\partial w_4} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial y_2} \cdot \frac{\partial y_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_4} = -(y - a_3) \cdot w_6 \cdot y_2(1 - y_2) \cdot x_2 \quad (18.19)$$

where we used that for a sigmoidal function f , its derivative is $f'(x) = f(x)(1 - f(x))$, which helped to write

$$\frac{\partial y_2}{\partial a_2} = \frac{\partial \varphi(a_2)}{\partial a_2} = \varphi(a_2)(1 - \varphi(a_2)) = y_2(1 - y_2) . \quad (18.20)$$

The derivation for $\partial \mathcal{J}/\partial b_1$, $\partial \mathcal{J}/\partial w_1$, and $\partial \mathcal{J}/\partial w_2$ follows along the same line and are summarized as

$$\frac{\partial \mathcal{J}}{\partial b_1} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial y_1} \cdot \frac{\partial y_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial b_1} = -(y - a_3) \cdot w_5 \cdot y_1(1 - y_1) \quad (18.21)$$

$$\frac{\partial \mathcal{J}}{\partial w_1} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial y_1} \cdot \frac{\partial y_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1} = -(y - a_3) \cdot w_5 \cdot y_1(1 - y_1) \cdot x_1 \quad (18.22)$$

$$\frac{\partial \mathcal{J}}{\partial w_2} = \frac{\partial \mathcal{J}}{\partial a_3} \cdot \frac{\partial a_3}{\partial y_1} \cdot \frac{\partial y_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_2} = -(y - a_3) \cdot w_5 \cdot y_1(1 - y_1) \cdot x_2 . \quad (18.23)$$

These are all possible “sensitivities” of the cost function w.r.t. the network parameters, and the backward pass is finished.

Update of a Weight in a Gradient Descent Approach

The above results can now be used in a [GD](#) method, where, e.g., the weight w_4 is updated according to

$$w_4^{\text{new}} = w_4^{\text{old}} - \eta \frac{\partial \mathcal{J}}{\partial w_4} , \quad (18.24)$$

where η is the learning rate. Note that if the algorithm is performed with multiple data records, then each record would result in a different weight increment. The weight update has to be done using the mean value of all weight increments. For weight w_j , this is then written as

$$w_j^{\text{new}} = w_j^{\text{old}} - \eta \frac{1}{n} \sum_{i=1}^n \frac{\partial \mathcal{J}(\mathbf{x}_i)}{\partial w_j} , \quad (18.25)$$

where n denotes the number of records. With this, all components for a complete training process are now available. The following Python implementation almost exactly uses the mathematical formulations.

18.3.3 A Complete Python Implementation

In the previous subsection, we derived the formulation for a forward and backward pass under the assumption that there is only one example in the dataset. What would change if, instead, we have n data records? The partial derivative operations will be “simultaneously” applied to all data records, but there are no vector–matrix products involved. Only for computing the cost function, we have to return to the formulation in Eq. 18.5. Therefore, we can directly implement the case of multiple data records using vectorization over data records. However, we will not generalize the formulation to arbitrary numbers of, e.g., input nodes or hidden layers but rather hard-code the network architecture.

Python Listing 18.1 shows a Python implementation of the formulations derived in Sect. 18.3.2, which will be briefly explained in what follows. We assume that the training data are provided in form of two feature vectors $\mathbf{x}_1$ and $\mathbf{x}_2$ together with the target vector $\mathbf{y}$, all of which are numpy arrays. The goal was to keep the implementation as readable as possible. For this we keep all weights and bias value in the sets $\mathbf{w}$ and $\mathbf{b}$ when functions are called. This requires that inside the function we again extract (or “unpack”) the individual weight and bias values from these two variables—and similar, when, e.g., the $dJdw1$, $dJdw2$ are returned. Using object oriented programming and classes could help to avoid this kind of “book-keeping overhead.”

- In line 3 we define the sigmoidal function. Due to the fact that we use numpy’s exponential function, it is possible to compute it for all elements of an array.
- Starting with line 6, we define a function for performing the forward pass. After extracting the individual weight and bias variables from $\mathbf{w}$ and $\mathbf{b}$, we can directly generate Python code from the mathematical equations. All operations work for $\mathbf{x}_1$ and $\mathbf{x}_2$ being scalars as well as numpy arrays. The individual weights and biases are always scalar values.
- Predicting a label for given weights, biases, and feature values in Line 16 consists just in a forward pass. The “_” in Line 15 are placeholders’ return values that are not required (but any other variable name would do as well).
- Doing the backward pass in Line 20 does not require anything new. The implementation is again identical to the mathematical formulation and works both for scalar and array data for $\mathbf{x}_1$ and $\mathbf{x}_2$ as all operations are performed element-wise.
- For updating the weights in Line 37 we again start by unpacking the weights, biases, and derivatives. Then, Eq. 18.25 is implemented and the updated weights and biases are returned

```

1 import numpy as np
2
3 def sigmoidal_function(x):
4     return 1 / (1 + np.exp(-x))
5
6 def forward_pass(W, B, X1, X2):
7     (w1, w2, w3, w4, w5, w6), (b1, b2, b3) = W, B
8
9     a1 = b1 + X1 * w1 + X2 * w2
10    a2 = b2 + X1 * w3 + X2 * w4
11    y1 = sigmoidal_function(a1)
12    y2 = sigmoidal_function(a2)
13    a3 = y1 * w5 + y2 * w6 + b3
14    return a1, a2, y1, y2, a3
15
16 def predict(W, B, X1, X2):
17     _, _, _, _, Y_pred = forward_pass(W, B, X1, X2)
18     return Y_pred
19
20 def backward_pass(W, B, X1, X2, Y):
21     w1, w2, w3, w4, w5, w6 = W
22     a1, a2, y1, y2, a3 = forward_pass(W, B, X1, X2)
23
24     dJdb1 = -(Y - a3) * w5 * y1 * (1 - y1)
25     dJdw1 = dJdb1 * X1
26     dJdw2 = dJdb1 * X2
27     dJdb2 = -(Y - a3) * w6 * y2 * (1 - y2)
28     dJdw3 = dJdb2 * X1
29     dJdw4 = dJdb2 * X2
30     dJdb3 = -(Y - a3)
31     dJdw5 = -(Y - a3) * y1
32     dJdw6 = -(Y - a3) * y2
33
34     dJdW, dJdB = (dJdw1, dJdw2, dJdw3, dJdw4, dJdw5, dJdw6), (dJdb1, dJdb2, dJdb3)
35     return dJdW, dJdB
36
37 def update_weights(W, B, dJdW, dJdB, learning_rate):
38     (w1, w2, w3, w4, w5, w6), (b1, b2, b3) = W, B
39     (dJdw1, dJdw2, dJdw3, dJdw4, dJdw5, dJdw6), (dJdb1, dJdb2, dJdb3) = dJdW, dJdB
40
41     w1 -= learning_rate * np.mean(dJdw1)
42     w2 -= learning_rate * np.mean(dJdw2)
43     w3 -= learning_rate * np.mean(dJdw3)
44     w4 -= learning_rate * np.mean(dJdw4)
45     w5 -= learning_rate * np.mean(dJdw5)
46     w6 -= learning_rate * np.mean(dJdw6)
47     b1 -= learning_rate * np.mean(dJdb1)
48     b2 -= learning_rate * np.mean(dJdb2)
49     b3 -= learning_rate * np.mean(dJdb3)
50
51     W_new, B_new = (w1, w2, w3, w4, w5, w6), (b1, b2, b3)
52     return W_new, B_new

```

Python Listing 18.1 Implementation of a network with two input nodes, two nodes in a hidden, layer and a single output node. See the text for more information. Note that contrary to all best practices in programming, we omitted docstrings, comments, type hints, and “sanity checks” to ensure that the code stays compact

With this we can now attempt to use the implementation for a concrete problem: classification using only the first two features of the “Iris dataset.” We import the dataset using the scikit-learn module (or the online resources of this book, or any other resources—as long as `x` contains the four features in columns and `y` is the single target variable) and then extract only those records that belong to the first two classes.

```
In [1]: import numpy as np
        from sklearn.datasets import load_iris

        iris = load_iris()
        X = iris["data"][:, :2]
        Y = iris["target"]

        # only use the first two labels 0 and 1
        X = X[Y < 2]
        Y = Y[Y < 2]
```

The data are split into a training and a testing dataset, using the function `train_test_split` that has already been introduced in Sect. 16.2.2.

```
In [2]: # split into training and testing data
        X_train, Y_train, X_test, Y_test = train_test_split(X, Y)

        # and for convenience define individual features X1 and X2
        X1_train = X_train[:, 0]
        X2_train = X_train[:, 1]
        X1_test = X_test[:, 0]
        X2_test = X_test[:, 1]
```

Next, we initialize the six weights with random values and set the bias values to zero. The number of epochs (i.e., the number of complete forward/backward passes) together with the learning rate are set

```
In [3]: W = 1e-3 * (rng.random(size=6) - 0.5)
        B = np.zeros(3)
        epochs = 600
        learning_rate = 0.2
```

with which now the gradient descent update iterations can be performed:

```
In [4]: mean_losses = []
        for epoch in range(epochs):
            Y_pred = predict(W, B, X1_train, X2_train)
            mean_losses.append(mean_loss(Y_train, Y_pred))

            dJdW, dJdB = backward_pass(W, B, X1_train, X2_train, Y_train)
            W, B = update_weights(W, B, dJdW, dJdB, learning_rate)

        print('MSE difference:', mean_losses[0], mean_losses[-1])
```

```
Out [4]: MSE difference: 0.26667596178013414 0.019281451533922658
```

Comparing the initial training MSE to the final training MSE suggests that at least the numerical scheme is stable and no undefined values or other errors occurred. Since the training dataset is two-dimensional, it is a good approach to visualize the data. Here, we show two different plot types. The first plot just shows the training *mean squared error* (MSE) as a function of the number of epochs:

```
In [5]: fig, ax = plt.subplots()
ax.plot(mean_losses)
ax.set(xlabel='number of epochs', ylabel='mean training loss (MSE)')
```

The second plot is a more complex plot. It is a heat map using `imshow`, which shows the training MSE evaluated at a grid of equidistant points of the feature space. This was superimposed with a scatter plot of the position of the training and testing data. Additionally, the decision boundary was shown where all target values smaller than 0.5 indicated class #1 and all values larger than 0.5 indicate class #2.

```
In [6]: fig, ax = plt.subplots()
ax.set(xlabel='X1', ylabel='X2');

# heatmap with contourplot (showing the decision boundary)
XX1, XX2 = np.meshgrid(np.linspace(4, 7.25), np.linspace(1.5, 4.6))
Y_pred = predict(W, B, XX1, XX2)
im = ax.imshow(Y_pred, origin='lower', vmin=-1, vmax=2, cmap='RdBu_r',
               extent=(XX1.min(), XX1.max(), XX2.min(), XX2.max()))
plt.colorbar(im)
ax.contour(XX1, XX2, Y_pred, levels=[0.5])

# scatter plot
ax.scatter(X1_train, X2_train, c=Y_train, cmap='RdBu_r', label='training')
ax.scatter(X1_test, X2_test, c=Y_test, cmap='RdBu_r', marker='.',
          label='testing')
ax.legend(ncols=2, loc='lower center')
```

The resulting plots are similar to those shown in Fig. 18.3 (plotting the loss curve for the testing follows the same path and is left as an exercise). Additionally, we also showed the evolution of the mean testing loss, which is left as an exercise for the reader.

The learning curve shows that after approximately 400 epochs, a stationary state is reached, both for training and testing. The testing MSE is slightly below the training MSE, but using different random splits does not result in significant

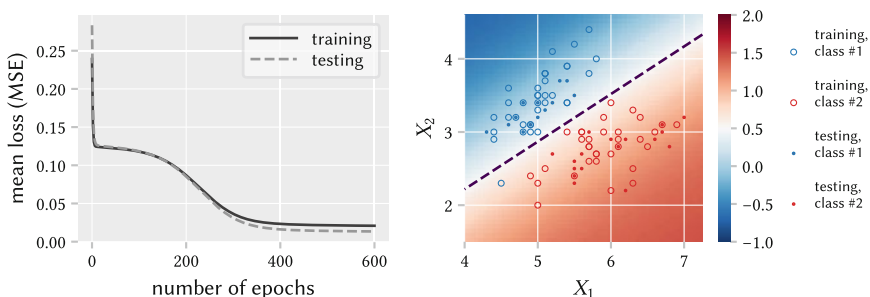


Fig. 18.3 Results from the network with one hidden layer for the first two features of the Iris dataset. The left panel shows the learning curve, the right panel shows the datasets together with the predicted value Y^{pred} and the decision boundary

difference. Here, a cross-validation could be useful as well. Visually inspecting the dataset reveals that the data points should be linearly separable; the decision boundary that was found by the network does this perfectly.

To quantify how well our model performs, we evaluate the metric of true positives, true negatives, false negatives, and false positives, as introduced in Sect. 11.8.1:

```
In [7]: TP = np.sum((Y_pred == 1) & (Y_test == 1))
        TN = np.sum((Y_pred == 0) & (Y_test == 0))
        FP = np.sum((Y_pred == 1) & (Y_test == 0))
        FN = np.sum((Y_pred == 0) & (Y_test == 1))

        print(f'TP: {TP}, FP: {FP}, TN: {TN}, FN: {FN}')
```

```
Out [7]: TP: 17, FP: 0, TN: 13, FN: 0
```

The value of 0.5 is the threshold above which the output is considered to be True. The result confirms what we already observed in the right panel of Fig. 18.3: all examples have been correctly predicted. Even though this comes as no surprise since the two classes are linearly separable, it is nonetheless a good validation of our network.

18.4 General Formulation of Backpropagation

In the last section, it became obvious that determining all chained partial derivatives of all layers is very tedious. It is also inflexible as for every new *network architecture* (i.e., the particular combination of different layers and nodes), the derivations have to be performed anew. An alternative way is to treat each layer as a basic building block that takes an input, processes the data, and returns an output. For example, for layer number k the input is $\mathbf{x}^{(k)}$ and the output is $\mathbf{y}^{(k)}$. For simplicity of the formulations, both vectors represent only a single data record and are thus row vectors. Also, we treat the activation functions as an individual layer, the *activation layer* (AL) while the layer where the summation is done is called *fully connected layer* (FCL).

This is also the way how current DL frameworks are designed. Therefore, we will go through these derivations in some more detail.

18.4.1 Forward Pass

Fully Connected Layer

We will now derive the formulations for the forward pass and start with the fully connected layer, which is layer number k . A sketch of the considered situation is shown in Fig. 18.4.

The forward pass works as expected: it takes as input the output from the previous layer (step 1. in Fig. 18.4), computes the values of the target variable (step 2. in

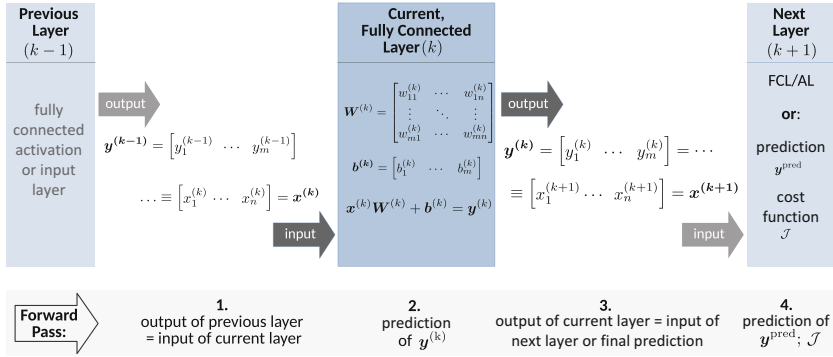


Fig. 18.4 Notations and visualization of the forward pass for a fully connected layer. Note that the sizes n and m of the input and output vectors can change from layer to layer even though we do not explicitly indicate this by subscripts such as (k)

Fig. 18.4), and provides this as the output to the next layer (step 3. in Fig. 18.4). If the next layer is the output layer, then this is the prediction of the network, and we additionally can evaluate the cost function (step 4. in Fig. 18.4).

What is new in this section is the notation of using superscripts to indicate the layer. Thereby, we only have to define each layer type ones and can then use them as basic building blocks for general networks. For example, the output of layer number $k-1$ is given by

$$\mathbf{y}^{(k-1)} = [y_1^{(k-1)} \dots y_m^{(k-1)}] \equiv [x_1^{(k)} \dots x_n^{(k)}] = \mathbf{x}^{(k)}, \quad (18.26)$$

which serves as the input of layer number k . The number of input nodes of each layer is denoted by n , while the number of output nodes of each layer is m . To be fully consistent, also the m and n would need a superscript to indicate the number of the layer; however, given that for m and n usually the context is clear and for the sake of readability, we will remember that, e.g., the number of input nodes n of a layer k can be different from the number of input nodes n of any other layer, and that writing n in fact would imply $n^{(k)}$ for the k -th layer.

The agglomeration in the layer is then given by

$$\mathbf{x}^{(k)} \mathbf{W}^{(k)} + \mathbf{b}^{(k)} = \mathbf{y}^{(k)} \quad (18.27)$$

where the matrix of weights and the vector of bias values of layer k are given by

$$\mathbf{W}^{(k)} = \begin{bmatrix} w_{11}^{(k)} & \dots & w_{1m}^{(k)} \\ \vdots & \ddots & \vdots \\ w_{n1}^{(k)} & \dots & w_{nm}^{(k)} \end{bmatrix} \quad \text{and} \quad \mathbf{b}^{(k)} = [b_1^{(k)} \dots b_m^{(k)}]. \quad (18.28)$$

The r -th component of $\mathbf{y}^{(k)}$ can then be written as

$$y_r^{(k)} = \begin{bmatrix} x_1^{(k)} & \dots & x_n^{(k)} \end{bmatrix} \begin{bmatrix} w_{1r}^{(k)} \\ \vdots \\ w_{nr}^{(k)} \end{bmatrix} + b_r^{(k)} = x_1^{(k)} w_{1r}^{(k)} + \dots + x_n^{(k)} w_{nr}^{(k)} + b_r^{(k)} . \quad (18.29)$$

Activation Layer

The formulation for the activation layer is rather straightforward, and all that needs to be done, is, to apply the activation function to each input of the layer. For example, for a layer number $k + 1$ it is

$$\mathbf{y}^{(i+1)} = \left[\varphi(x_1^{(i+1)}), \dots, \varphi(x_n^{(i+1)}) \right] = \varphi \left(\mathbf{x}^{(i+1)} \right) , \quad (18.30)$$

where the function φ is applied to each element of $\mathbf{x}$ individually. $\mathbf{x}$ is the input of the activation layer, which is typically given by the output of a fully connected layer. In an activation layer, $\mathbf{y}$ has always the same dimensions as $\mathbf{x}$.

Cost Function

The forward propagation step is then repeated for each layer, until we arrive at the output layer for which we obtain the predicted output $\mathbf{y}$. With this, the cost function can be evaluated; as we only have a single data record, it is simply

$$\mathcal{J}(\mathbf{y}; \mathbf{y}^{\text{train}}) = \frac{1}{2m} \sum_{i=1}^m \left(y^{\text{train}}_i - y^{\text{pred}}_i \right)^2 \quad (18.31)$$

18.4.2 Backward Pass

The backward pass is more complicated than the forward pass and requires the computation of a number of derivatives. The strategy is to establish a backward pass for one layer, which can then be generalized to more complex networks by connecting such layers.

Our point of departure for the derivation is the output layer where we already computed the cost function. The goal is now to use the established framework of a gradient descent method, which is supposed to find values of *all* weights and *all* bias values of the N_{layers} layers in the network such that the cost function is minimized,

$$\text{Find } \left\{ \mathbf{w}^{(k)}, \mathbf{b}^{(k)} \right\}_{k=1 \dots N_{\text{layers}}} \text{ such that } \mathcal{J} \rightarrow \text{MIN} . \quad (18.32)$$

The numerical approach is the same as before: we calculate the sensitivity of the cost function w.r.t. all weights and bias values,

$$\frac{d\mathcal{J}}{dw_{ij}^{(k)}} \quad \text{and} \quad \frac{d\mathcal{J}}{db_j^{(k)}} , \quad (18.33)$$

and use this to define an update increment for all weights and bias values. While computing these derivatives manually is possible in case that there is only one fully connected layer (as we saw in the previous chapter), for networks with several layers, this is more difficult as one layer depends on the output of the previous layer, and we thus have an implicit dependency of the cost function on the weights. This is where the *chain rule* for a function that implicitly depends on a third variable through two variables will come into play:

Definition 18.1. Chain Rule If f is a function of two variables, $f(y_1, y_2)$, and each of the y_1 and y_2 are again functions of a variable x , then it is

$$\frac{df}{dx} = \frac{\partial f}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial f}{\partial y_2} \frac{\partial y_2}{\partial x} . \quad (18.34)$$

In our case, the function under consideration is the cost function $\mathcal{J}$, and we are looking for the derivatives w.r.t. the weights and biases, see Eq. 18.33. However, we are only able to obtain the derivatives w.r.t. to the final output $y^{(k)}$, because this is the only variable that shows up in Eq. 18.31. What other information could be used? We already know that $\mathbf{y}$ implicitly depends also on the weights and bias values. Thus, we can write the first term from Eq. 18.33 using the chain rule as

$$\begin{aligned} \frac{d\mathcal{J}}{dw_{ij}^{(k)}} &= \frac{\partial \mathcal{J}}{\partial y_1^{(k)}} \frac{\partial y_1^{(k)}}{\partial w_{ij}^{(k)}} + \cdots + \frac{\partial \mathcal{J}}{\partial y_m^{(k)}} \frac{\partial y_m^{(k)}}{\partial w_{ij}^{(k)}} \\ &= \sum_{r=1}^m \frac{\partial \mathcal{J}}{\partial y_r^{(k)}} \frac{\partial y_r^{(k)}}{\partial w_{ij}^{(k)}} . \end{aligned} \quad (18.35)$$

Each of the summands in Eq. 18.35 consists of two factors. The first terms *only* can be easily computed for the final layer using the definition of the cost function. For example, for the r -th component of $\mathbf{y}$ we get:

$$\frac{\partial \mathcal{J}}{\partial y_r^{(k)}} = -\frac{1}{m} \left(y^{\text{train}}_r - y_r \right) , \quad (18.36)$$

where m is the number of output nodes which is identical to the number of elements of $\mathbf{y}$. For all other layers, we first have to propagate the error from the output layer to the respective layer of interest. For doing so, the chain rule comes in handy again,

and we can write:

$$\frac{d\mathcal{J}}{dx_i^{(k)}} = \sum_{r=1}^m \frac{\partial \mathcal{J}}{\partial y_r^{(k)}} \frac{\partial y_r^{(k)}}{\partial x_i^{(k)}} \quad \text{or} \quad \frac{d\mathcal{J}}{d\mathbf{x}^{(k)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \frac{\partial \mathbf{y}^{(k)}}{\partial \mathbf{x}^{(k)}}. \quad (18.37)$$

Next, we realize that the output of layer $(k - 1)$ serves as the input of layer k . With this we are now able to rewrite Eq. 18.37 as

$$\frac{d\mathcal{J}}{d\mathbf{y}^{(k-1)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \frac{\partial \mathbf{y}^{(k)}}{\partial \mathbf{x}^{(k)}}, \quad (18.38)$$

which also very clearly shows where “back-propagation” got its name from: we start from the end of the network and iteratively compute the derivative of the cost function w.r.t. the weights for layer $k - 1$ using the already known value from the previous layer k . Thereby, the change of the overall error is moving backward through all layers.

For the following derivations, we again differentiate between a fully connected layer and an activation layer.

Fully Connected Layer

First, we continue with the previous expression, Eq. 18.38. For a fully connected layer, we can specify $\partial \mathbf{y} / \partial \mathbf{x}$

$$\frac{d\mathcal{J}}{d\mathbf{y}^{(k-1)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \mathbf{W}^{(k)}. \quad (18.39)$$

Next, the second term in Eq. 18.35 still requires to be determined. We start from explicitly rewriting them in terms of the components of Eq. 18.29:

$$\begin{aligned} \frac{\partial y_1^{(k)}}{\partial w_{ij}^{(k)}} &= \frac{\partial}{\partial w_{ij}^{(k)}} \left(x_1^{(k)} w_{11}^{(k)} + x_2^{(k)} w_{21}^{(k)} + \dots + x_n^{(k)} w_{n1}^{(k)} + b_1^{(k)} \right) \\ \frac{\partial y_2^{(k)}}{\partial w_{ij}^{(k)}} &= \frac{\partial}{\partial w_{ij}^{(k)}} \left(x_1^{(k)} w_{12}^{(k)} + x_2^{(k)} w_{22}^{(k)} + \dots + x_n^{(k)} w_{n2}^{(k)} + b_2^{(k)} \right) \\ &\vdots \\ \frac{\partial y_m^{(k)}}{\partial w_{ij}^{(k)}} &= \frac{\partial}{\partial w_{ij}^{(k)}} \left(x_1^{(k)} w_{1m}^{(k)} + x_2^{(k)} w_{2m}^{(k)} + \dots + x_n^{(k)} w_{nm}^{(k)} + b_m^{(k)} \right). \end{aligned} \quad (18.40)$$

All derivatives are zero except for the only term containing a w_{ij} :

$$\frac{\partial y_j^{(k)}}{\partial w_{ij}^{(k)}} = \frac{\partial}{\partial w_{ij}^{(k)}} \left(x_i^{(k)} w_{ij}^{(k)} \right) = x_i^{(k)} \quad (18.41)$$

which is the expression that contains the j -th component of $\mathbf{y}$. Hence, we can simplify Eq. 18.35 to

$$\frac{d\mathcal{J}}{d\mathbf{w}_{ij}^{(k)}} = \frac{\partial \mathcal{J}}{\partial y_j^{(k)}} x_i^{(k)}, \quad (18.42)$$

which can also be written as the outer product of two vectors

$$\frac{d\mathcal{J}}{d\mathbf{W}^{(k)}} = \mathbf{x}^{(k)} \otimes \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \equiv \left(\mathbf{x}^{(k)} \right)^\top \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}}, \quad (18.43)$$

where the last term rewrites the outer product symbol $\otimes$ as the product of a column vector with a row vector, resulting in a matrix. The derivation for the derivative w.r.t. the bias follows nearly the identical path, and we end up with

$$\frac{\partial y_j^{(k)}}{\partial b_j^{(k)}} = 1, \quad (18.44)$$

such that the sensitivity of the cost function w.r.t. the bias is given by

$$\frac{d\mathcal{J}}{d\mathbf{b}^{(k)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}}. \quad (18.45)$$

We can now summarize all details for a back-propagation step for a fully connected layer in Algorithm 18.1:

Algorithm 18.1: Backpropagation Step of a Fully Connected Layer



The following are the required steps for backpropagation:

1. If the layer $k + 1$ is the final, output layer, compute

$$\frac{\partial \mathcal{J}}{\partial \mathbf{y}} = -\frac{1}{m} (\mathbf{y}^{\text{train}} - \mathbf{y}); \quad (18.46)$$

otherwise just use the value $\partial \mathcal{J} / \partial \mathbf{y}^{(k+1)}$ from the previous layer in step 2.

2. Propagate the error from layer $k + 1$ to layer k :

$$\frac{d\mathcal{J}}{d\mathbf{y}^{(k)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k+1)}} \mathbf{W}^{(k)}. \quad (18.47)$$

3. Compute the sensitivity of the cost w.r.t. the weights and bias values of the current layer

$$\frac{d\mathcal{J}}{d\mathbf{w}^{(k)}} = \mathbf{x}^{(k)} \otimes \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \quad \text{and} \quad \frac{d\mathcal{J}}{d\mathbf{b}^{(k)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \quad (18.48)$$

4. Update the weights and biases using the GD methods

$$\mathbf{w}^{(k)} \leftarrow \mathbf{w}^{(k)} - \frac{\eta}{N} \frac{d\mathcal{J}}{d\mathbf{w}^{(k)}} \quad \text{and} \quad \mathbf{b}^{(k)} \leftarrow \mathbf{b}^{(k)} - \frac{\eta}{N} \frac{d\mathcal{J}}{d\mathbf{b}^{(k)}} \quad (18.49)$$

with the learning rate η and N the number of data records.

5. Continue to the next layer, $(k) \Leftrightarrow (k - 1)$.

Activation Layer

The back-propagation of the errors for one activation layer is significantly easier. We again start from Eq. 18.38 for which we need to specify $\partial \mathbf{y} / \partial \mathbf{x}$. Recall the definition of the forward pass of the activation layer in Eq. 18.30, which states that the output is obtained from applying the activation function on each of the entries of $\mathbf{x}$ individually. The derivative then reads

$$\frac{d\mathcal{J}}{d\mathbf{y}^{(k-1)}} = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \frac{\partial \varphi(\mathbf{x}^{(k)})}{\partial \mathbf{x}^{(k)}} \quad (18.50)$$

$$= \left[\frac{\partial \mathcal{J}}{\partial y_1^{(k)}} \frac{\partial \varphi(x_1^{(k)})}{\partial x_1^{(k)}}, \dots, \frac{\partial \mathcal{J}}{\partial y_n^{(k)}} \frac{\partial \varphi(x_n^{(k)})}{\partial x_n^{(k)}} \right] = \frac{\partial \mathcal{J}}{\partial \mathbf{y}^{(k)}} \cdot \quad (18.51)$$

Thus, the sensitivity of the error w.r.t. the output is not changed by the activation layer. Note that the product in Eq. 18.50 is the element-wise (or HADAMARD) product as both terms result in row vectors. Nothing else needs to be done here as there are no trainable weights or biases.

18.5 Python Implementation and Example for the Fully Connected Network

The algorithm and the data structure for this network are already fairly complex. As a consequence, the Python code is also getting more lengthy and requires vectorization of the numpy operations. Therefore, we are approaching a point at which showing code and commenting it are no longer feasible in this text. Nonetheless, we will do this for the fully connected network, and, based on this, explain the design strategy and the implementation details. For the Python

implementation of all subsequent network architectures and the corresponding DL experiments, we refer the reader to the accompanying website.

18.5.1 Python Implementation

The Python implementation can be best done in an object-oriented approach using classes and inheritance,² because each activation layer of a network has always the same structure and needs to perform the same computations as derived above. Additionally, different layers must be able to keep track of their “state,” governed by the input values and, in the case of a fully connected layer, also of the values of weights and biases. Thus, each fully connected layer is an instantiation of the same class. The same also holds for the activation layer. Since we want to ensure that all layers should have the same “structure,” in terms of input and output of the functions (or rather methods, as it is called in the context of a class), we create an *abstract base class* from which both the activation layer and the fully connected layer are derived:

```
In [1]: from abc import ABC # For implementing an abstract base class (ABC)
import numpy as np

class NNLayer(ABC):
    def __init__(self):
        self.x = np.empty()
        self.y = np.empty()

    def feed_forward(self, x):
        raise NotImplementedError("Any layer needs to implement the \
                                feed_forward method.")

    def backward_propagation(self, dJdy, learning_rate):
        raise NotImplementedError("Any layer needs to implement the \
                                backward_propagation method.")
```

Thereby, we ensure that any derived class implements the feed-forward method and the backward propagation method.

We now begin with the concrete implementation of the fully connected layer, which we will explain in two steps. `FullyConnectedLayer` derived from the abstract base class `NNLayer` and therefore inherits all attributes and methods. The `__init__` method initializes the random number generator using the integer value `seed` as a seed. It also initializes the weights and biases with random values. The “state” of the class (i.e., the member variables `x` and `y`) is set during the forward pass. Note, that all numpy operations are vectorized and need to have correct dimensions; for brevity, we do not ensure this.

² We are fully aware that not everyone is familiar with object-oriented programming, and, so far, we entirely avoided it. However, the object-oriented code is, in our opinion, much more readable than a code written with functional programming. For those who are looking for an introduction, [3] is one out of many good introductions.

```
In [2]: class FullyConnectedLayer(NNLayer):
    def __init__(self, n_inputs, n_outputs, seed=None):
        rng = np.random.default_rng(seed)
        self.weights = rng.random(size=(n_inputs, n_outputs)) - 0.5
        self.biases = rng.random(size=(1, n_outputs)) - 0.5

    def feed_forward(self, x):
        self.x = np.atleast_2d(x)
        self.y = np.dot(self.x, self.weights) + self.biases
        return self.y

    def backward_propagation(self, dJdy, learning_rate): ...
```

The implementation of the backward propagation is shown in the next cell. The class method implements the sensitivity from step 3 as well as the update step 4 from the above algorithm Algorithm 18.1. If we would not implement these methods, the corresponding methods of the base class (cf. In [1]:) would be called. This causes an exception to be raised and the program stops.

```
In [2]: class FullyConnectedLayer(NNLayer):
    def __init__(self, n_inputs, n_outputs, seed=None): ...
    def feed_forward(self, x): ...

    def backward_propagation(self, dJdy, learning_rate):
        dJdW = np.dot(self.x.T, dJdy)
        dJdb = np.dot(np.ones(self.x.shape[0]), dJdy)
        self.weights -= learning_rate * dJdW
        self.biases -= learning_rate * dJdb

        dJdy_prev = np.dot(dJdy, self.weights.T)
        return dJdy_prev
```

As a preparation for the activation layer, we now define a activation function and its derivative.

```
In [3]: def tanh_function(x):
    return np.tanh(x)

def tanh_derivative(x):
    return 1 - np.tanh(x) ** 2
```

We are now able to implement the activation layer, which is again derived from the abstract base class `NNLayer`. The specific activation function is hard-coded in the `__init__` method, because all activation layers should use the same activation function. The feed-forward and the backward propagation methods follow the mathematical form.

```
In [4]: class ActivationLayer(NNLayer):
        def __init__(self):
            self.phi = tanh_function
            self.dphidx = tanh_derivative

        def feed_forward(self, x):
            self.x = x
            self.y = self.phi(self.x)
            return self.y

        def backward_propagation(self, dJdy, learning_rate):
            return self.dphidx(self.x) * dJdy
```

For later use during the training, we now implement the cost function and its derivative based on the [MSE](#).³

```
In [5]: def MSE(y_true, y_pred):
        return np.mean((y_true - y_pred) ** 2)

        def dMSE_dy(y_true, y_pred):
            return 2 / y_pred.size * (y_pred - y_true)
```

We are now ready to implement the `Network` class, which contains functionality for assembling the network, for training, and for predicting. Note that the following three code snippets always show the same class but hide the methods that are not being explained, indicated by the three dots. We start by storing a reference to the cost function and its derivative, which are to be defined during creation of the network. All layers are simply stored in a list by the method `add_layer`.

```
In [6]: class Network:
        def __init__(self, cost_function, derivative_of_cost):
            self.layers = []
            self.cost_function = cost_function
            self.derivative_of_cost = derivative_of_cost

        def add_layer(self, layer):
            self.layers.append(layer)

        def train(self, X_train, Y_train, epochs, learning_rate): ...

        def predict(self, X): ...
```

The method that does the actual training needs some more explanations. The training method takes the training dataset, the number of training epochs (i.e., the number of iterations), and the learning rate. During one epoch, it iterates through the whole training dataset and for each pair `x_train, y_train`, it performs the forward propagation through *all* layers. The line `y_pred = x_train` might need a short explanation: in the following `for` loop, the forward propagation is done by using the output of the previous as the input of the current layer. This procedure

³ Sometimes, a factor of 2 be found in the [MSE](#). However, this factor does not affect the training behavior if the learning rate is scaled accordingly.

is sequentially done using the variable `y_pred`, which is why `y_pred` needs to be properly initialized with the output of the “previous” layer, the inputs `x_train`. At the end of this sequence, we obtain the cost function for the prediction of the output layer.

Next, the errors are back-propagated through all layers, which adjusts all weights and biases (see the back-propagation method in the above code cell “In [2]:”). At the end of each epoch, the value of the cost function is stored in the list `all_costs`, which will be returned at the end of the training. It then can be used, e.g., for plotting the training error.

```
In [6]: class Network:
        def __init__(self, cost_function, derivative_of_cost): ...
        def add_layer(self, layer): ...

        def train(self, X_train, Y_train, epochs, learning_rate):
            all_costs = []
            for i in range(epochs):
                cost = 0
                for x_train, y_train in zip(X_train, Y_train):
                    # compute prediction through forward propagations
                    y_pred = x_train
                    for layer in self.layers:
                        y_pred = layer.feed_forward(y_pred)

                    cost += self.cost_function(y_train, y_pred)

                    # perform backprop of errors
                    error = self.derivative_of_cost(y_train, y_pred)
                    for layer in reversed(self.layers):
                        error = layer.backward_propagation(error,
                                                            learning_rate)

                all_costs.append(cost / X_train.shape[0])
            return all_costs

        def predict(self, X): ...
```

In this part of the code, the variable `all_costs` is a list of the cost values for each epoch that can then be used for plotting. Since the training sometimes takes longer, it can be useful to get some feedback about the progress. The Python package `tqdm` is very useful for this purpose. For example, in a jupyter notebook, after importing `tqdm` with `from tqdm.notebook import trange`, you can replace `range` with `trange` and then will see a continuously updated progress bar.

The last method of the `Network` class is used for making predictions. It takes a feature matrix which, as usual, has records in rows, and each column is a feature. We iterate over all records and for each of them compute the output by a complete forward pass. For consistency, the resulting prediction is returned as a numpy array and not as a list.


```
In [6]: class Network:
        def __init__(self, cost_function, derivative_of_cost): ...
        def add_layer(self, layer): ...
        def train(self, X_train, Y_train, epochs, learning_rate): ...

        def predict(self, X):
            Y = []
            for x in X:
                y = x
                for layer in self.layers:
                    y = layer.feed_forward(y)
                Y.append(y)
            return np.array(Y)
```

With this, our implementation is finished, and we can perform a short test.

18.5.2 Application: Identification of Handwritten Digits

We use the dataset of handwritten digits, DS-2-small (UCI Handwritten Digits), and seek to find the labels 0...9 for each image in an supervised learning approach. We start by importing the data. Note that the dataset already contains the “flattened” images, i.e., each $N \times N$ image array was converted into a long one-dimensional array of $N \times N$ elements. We then perform *one-hot encoding* (OHE) (cf. Sect. 16.1.1) such that we can have an output layer with 10 units. The dataset is then split into a training and a testing portion with the function introduced in Sect. 16.2.2.

```
In [7]: from sklearn.datasets import load_digits
        digits = load_digits()
        X = digits.data / 16. # Scale the feature value range to [0, 1]

        # Target variable: One-hot-encoding: map each number of the interval [0,9]
        # 9 zeros and a one, such that, e.g., 2 is mapped to [0, 0, 1, 0, 0, 0, 0
        Y = digits.target
        Y = np.eye(10, dtype=int)[Y]

        X_train, Y_train, X_test, Y_test = \
            train_test_split(X, Y, fraction=0.7, seed=None)
```

We can now assemble the network. We chose an architecture consisting of as many input nodes as there are features, which is why in the code we use `X_train.shape[-1]`. These are 64 nodes, since the images contain $8 \times 8 = 64$ pixel. The hidden layer has 80 units. The next fully connected layer reduces the number of nodes to 40, the next but one reduced them to 10, which are the number of output nodes, consistent with the one-hot encoding. Finally, we train the model for 200 epochs and with a learning rate of 0.1. These are values that were experimentally found. A good strategy is to start with a few epochs and a learning rate in between 0.001 and 0.1. If the resulting MSE (shown as output of the last cell) does not blow up, then one could increase the number of epochs or increase the learning rate.

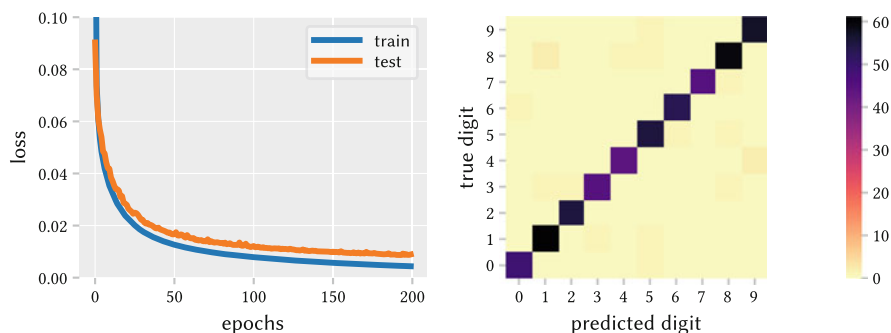


Fig. 18.5 Classification of the handwritten digits dataset using the implementation of the fully connected network. The left panel shows the training and testing error, the right panel shows the confusion matrix

```
In [8]: nn = Network(MSE, dMSE_dy)
nn.add_layer(FullyConnectedLayer(X_train.shape[-1], 80))
nn.add_layer(ActivationLayer())
nn.add_layer(FullyConnectedLayer(80, 40))
nn.add_layer(ActivationLayer())
nn.add_layer(FullyConnectedLayer(40, 10))
nn.add_layer(ActivationLayer())

train_mse = nn.train(X_train, Y_train, epochs=200, learning_rate=0.01)
train_mse[0], train_mse[-1]
```

```
Out [8]: (0.35482118936344176, 0.0018668214460151819)
```

There are (at least) two very useful methods to investigate the performance of the training. The first is to simply plot the training and testing error, as shown in Fig. 18.5.

It is important to ascertain that not only the training error decreases but also that the testing error decreases. With the above implementation, only the training error can directly be plotted. The testing MSE is obtained by using the weights and biases of each epoch for making predictions on the testing dataset. This is left as an exercise.

Creating the confusion matrix comes down to counting those digits that got correctly predicted. In the next Python cell, `yp` is an array of 10 entries (one for each digit, i.e., output node). Finding the one that has the largest value gives us the respective digit. This is done, using the `np.argmax` function.

```
In [9]: confusion_matrix = np.zeros((10, 10), dtype=int)
for yt, yp in zip(Y_test, Y_pred):
    pred_digit = np.argmax(yp)
    true_digit = np.argmax(yt)
    confusion_matrix[true_digit, pred_digit] += 1
```

We can now plot the confusion matrix, as shown in the next Python cell, where we used `imshow` to plot the two-dimensional 10×10 array.

```
In [10]: fig, ax = plt.subplots()
         im = ax.imshow(confusion_matrix, cmap='magma_r', origin='lower')
         plt.colorbar(im)
         ax.set(xlabel='predicted digit', ylabel='true digit',
               xticks=range(10), yticks=range(10));
```

We observe that both the train as well as the test loss decrease quite smoothly. The test loss is only slightly higher than the train loss. Both losses continue to very slowly decrease. The classification accuracy (for this particular random split into training and testing data) is after 200 epochs already at 96.5%, and this doesn't increase significantly even if we train for another 1000 epochs. The confusion matrix also shows a very good pattern, and only few images are misclassified. We observe that the two most misclassified digits are the 4 (which is identified as a 9) and the 8 (which is identified as a 1 but also as a 4 or 5). Quite surprising, on a first look, this agree only partially with the results from the principal component analysis in Fig. 15.9. However, the dataset investigated there was the full MNIST dataset DS-2 (MNIST) and not the DS-2-small (UCI Handwritten Digits) used here. Furthermore, there are only very few images misclassified at all, and these might be among the extreme values, hidden behind other data points in Fig. 15.9. There is an exercise that gives some hint on how to investigate this observation, cf. Exercise 18.11.

Machine Learning vs. Deep Learning



The above example also demonstrates a key difference between DL and classical ML: while in classical (or statistical) ML we need features, DL does not require features. We could simply use every pixel of a whole image as the input.

18.6 Further Concepts and Techniques

The fully connected neural network introduced above is already quite powerful and can perform complex tasks that would be difficult to achieve with classical ML methods. Nonetheless, there are a number of measures that can be implemented, in order to enhance the performance. We will now introduce some of them briefly.

18.6.1 Mini-Batches

There are two obvious ways for calculating the loss: we can either calculate it for one data record, or we could calculate it for the whole dataset. The former would

not help to improve the performance w.r.t. the *whole* dataset and would result in very unstable gradients. The latter may take a long time and require a huge amount of memory. A *min-batch* is a compromise between the two extremes. A mini-batch contains only a subset of the whole training data, with the number of contained data records called the *batch size*. The batch size is a hyperparameter to the model. During training, all mini-batches are then processed, until the whole training dataset is covered. The step-by-step algorithm is shown in Algorithm 18.2.

Algorithm 18.2: Training with Mini-batches



For the use of mini-batches, the following steps need to be taken:

1. shuffle the training dataset
2. split the training dataset such that each “fold” has as many records as given by the mini-batch size.
3. pick the first mini-batch
4. do a forward and backward pass simultaneously with all data of the current mini-batch
5. sum up the average contribution of the training cost for the mini-batch
6. choose the next mini-batch and go to step 4 until all mini-batches are processed.

Choosing a good batch size is essential for an efficient and accurate training process. This is another parameter that we, as the DL modeler, need to fix. Exercise 18.7 considers this aspect further.

18.6.2 Batch Normalization

When mini-batches are used, each of them is described by different summary statistics. *Batch normalization* is a method that centers and normalizes the data of a mini-batch and does so for each (or at least most of the) layers [4]. This is done just before the input is given to the activation functions, which seems to reduce fluctuations in the hidden layers activations. In practice, batch normalization is added in form of an extra layer. By *centering and normalizing*, we denote the transformation,

$$\hat{\mathbf{x}}^{(k)} = \frac{\mathbf{x}^{(k)} - \mu_{\mathcal{B}}}{\sigma_{\mathcal{B}}}, \quad (18.52)$$

where $\mathcal{B}$ denotes the minibatch under consideration and $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}$ are the respective values of the mean and standard deviation.

However, such a normalization would imply that from a sigmoidal activation function only the linear regime would be used. Thus, two additional parameters are

introduced for each layer k , the scale $\gamma^{(k)}$ and the shift of the normalized vector $\beta^{(k)}$ with

$$\tilde{\mathbf{x}}^{(k)} = \gamma^{(k)} \hat{\mathbf{x}}^{(k)} + \beta^{(k)} . \quad (18.53)$$

The two new parameter are not minibatch-specific and are learned along with the other weights and biases of the model. This transformation even allows to approximately undo the centering and normalization, if this would be the optimal approach. However, the new parameter are directly able to “tune” the variance and the mean, which are parameters that are otherwise only indirectly accessible by the network. This can significantly change the training behavior.

While there are still some speculations about the mathematical details of why this works, there is significant evidence that batch normalization strongly improves the performance of a network. In particular, the learning rate can be chosen much more generous.

18.6.3 Weight Initialization

So far, we have always initialized the weights with small random numbers, e.g., in the above example, taken from a uniform distribution with a range of $[-0.5, 0.5]$. We have already seen in Sect. 12.3.7 that during a minimization, such as (stochastic) gradient descent, the initial values can make a difference. In a deep neural net, the initialization of the weights may have a different influence depending on which kind of activation function is used. This also was recognized by XAVIER GLOROT and coworker [1], who suggested to scale the uniform distributions when using a linear activation. That strategy is known as the *Xavier initialization* for which the weights w are sampled from the following symmetric distribution:

$$w \sim \text{U} \left(-1/\sqrt{n}, +1/\sqrt{n} \right) , \quad (18.54)$$

where n is the number of input nodes and $\text{U}(a, b)$ denotes the uniform distribution within the limits a and b . As a consequence of the dependency on the number of input nodes, the range of the random values is larger for smaller n . Figure 18.6 shows the distribution of random samples as a function of the number of input nodes.

A variant of this initialization is the *normalized Xavier initialization*, which additionally takes the number of outputs m into account. The number of outputs are usually identical to the number of units in the layer under consideration. Then, the weights w are sampled from the following distribution:

$$w \sim \text{U} \left(-\sqrt{6}/\sqrt{n+m}, +\sqrt{6}/\sqrt{n+m} \right) . \quad (18.55)$$

The two variants of the Xavier initialization work well for sigmoidal and tanh activation functions (even though, the former is only rarely used, today).

Fig. 18.6 Xavier initialization: the range of the random uniform distribution is chosen according to the number of input nodes. For each value of n , 40 random values are drawn

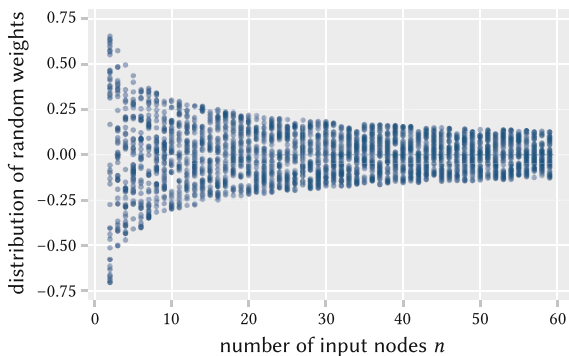
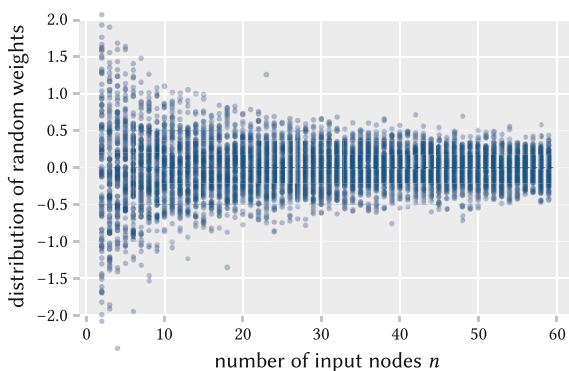


Fig. 18.7 He initialization: as opposed to the Xavier initialization, here a normal distribution is chosen to be sampled from. For each value of n , 100 random values are drawn



Words of advice



The two variants of the Xavier initialization do not work well together with the ReLU activation.

If the ReLU activation function is used, then the alternative formulation, introduced by KAIMING HE [2] is preferred. In this case, the random weight values are drawn from a centered normal distribution:

$$w \sim \mathcal{N}(\mu = 0, \sigma = \sqrt{2/n}) \quad (18.56)$$

Figure 18.7 shows the distribution of random weight values as a function of the number of input nodes.

Note, that the range of the weight values for small values of n is much larger than for the Xavier initialization. For larger n , the range of the values stays approximately constant (at least over a reasonable numbers of input nodes).

18.7 Less Is More: The Concept of Dropout

A serious problem of networks with large numbers of weights is overfitting. This applies in particular to fully connected networks, which, by construction, contain a maximum number of possible weights and thereby tend to be slow during training. *Dropout* is a technique for remedying this, first published in 2014 by Srivastava et al. [9]. The idea of dropout is to randomly drop nodes from the network and also to remove the respective connections. Such a kind of training is then performed for a number of “thinned” networks, each of which uses a random dropout. During testing, a fully connected network is used that represents the average of the trained networks (according to the probability with which a unit is found in one of the thinned realizations). Figure 18.8 shows a sketch of a network with two hidden layers.

The original publication [9] proposed to additionally include the units of the input layer in the set of those that can be randomly removed (they proposed a retention rate of 50% for the units in hidden layers and 80% (i.e. only every fifth unit is dropped) for those in the input layers). There are, however, situations where thinning the input layer might not be the most appropriate approach. For example, if the input consists of one-hot encoded categorical data, in cases where the number of inputs are very small, or if each input carries a unique and significant amount of information, one might want to keep all input nodes.

Why is the dropout technique useful? The quick answer is that it adds a new type of randomness to the training process, which is, generally speaking, always beneficial against overfitting. Part of the explanation is also that removing random units and using such network realizations in an ensemble, acts as a *stochastic regularization*. On a technical level, dropout is implemented by sampling, e.g., from a BERNOULLI distribution (cf. Sect. 10.3) and using the result to determine if a unit is taken into account during forward propagation. E.g., for layer k and the i -th unit

$$r_i^{(k)} \sim \text{Bernoulli}(p) \quad (18.57)$$

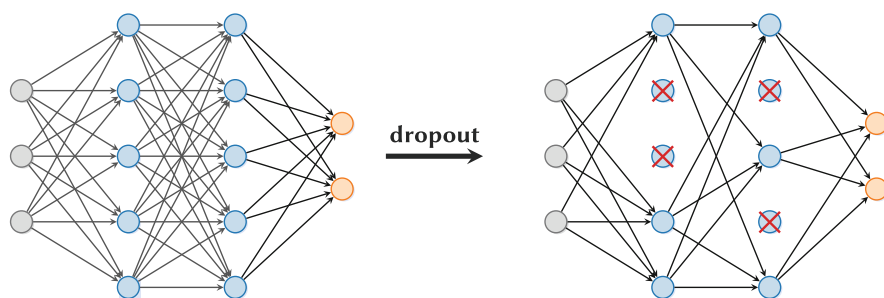


Fig. 18.8 “Drop out” transforms a fully connected network into a network where not all nodes are fully connected. This is achieved by randomly removing nodes from the network. Here, this was only applied to the hidden layers

$$\tilde{\mathbf{x}}^{(k)} = \mathbf{r}^{(k)} \odot \mathbf{x}^{(k)} \quad (18.58)$$

$$y_i^{k+1} = \mathbf{w}_i^{(k+1)} \cdot \tilde{\mathbf{x}}^{(k)} + b_i^{(k+1)} \quad (18.59)$$

where the parameter p of the BERNOULLI distribution is chosen according to the desired retention rate. Thus, $r_i^{(k)}$ effectively “switches” units on or off, ensuring that the desired percentage of units is disabled. The thinned output of this layer is used as input for the following layer. Effectively, the dropout network is one of the 2^n possible “sample” of the fully connected network, with n being the number of units. Ideally, one would then like to obtain a sample average of all 2^n networks. As this is usually unfeasible due to computational cost, the solution is that during testing, each weight obtained during training, is scaled by the probability p , and only a *single* fully connected network is used.

In terms of an implementation, taking out individual units amounts to removing the respective rows and columns from the linear systems of equations. This may significantly reduce the size of the minimization problem. Furthermore, all the other techniques used in “regular” networks, such as momentum and weight decay, seem to work nicely for dropout networks as well. However, keep in mind that this requires that the network is adjusted during the testing phase.

Therefore, for implementation reasons, often a different type of dropout is used, the *inverted dropout*. There, the scaling by the probability is changed to a scaling during the training, but with the inverse probability $1/p$.

When the dropout strategy was developed around 2012, it gave the development and the performance of DL method a significant boost; it is one of the major contributors that helped DL to become able to solve highly complex problems, sometimes even with super-human performance.

18.8 Example: Microstructure Classification and Property Prediction

As an example, we use the MDS-2 light (Ising Model, small images) dataset for the task of predicting if an image with a particular microstructure corresponds to a temperature that is above or below the Curie temperature (cf. Sect. 4.3 for further details). For this, we define a network that consists of a fully connected layer that takes the input (the image in form of a “flattened” vector), which is reduced to 128 output nodes. The following layer has 128 input and 64 output nodes, 64 and 32 input/output nodes, and finally 32 input and 2 output nodes that give the class label of the image for the one-hot encoding. Thus, the model has altogether approximately 43,000 weights to be determined. Figure 18.9 shows the model performance.

We observe that the model reaches a prediction accuracy of around 80%. Comparing this to the baseline of $\approx 50\%$ for a binary classification with approximately balanced classes (cf. Sect. 16.3), this represents a good (but not a great) accuracy.

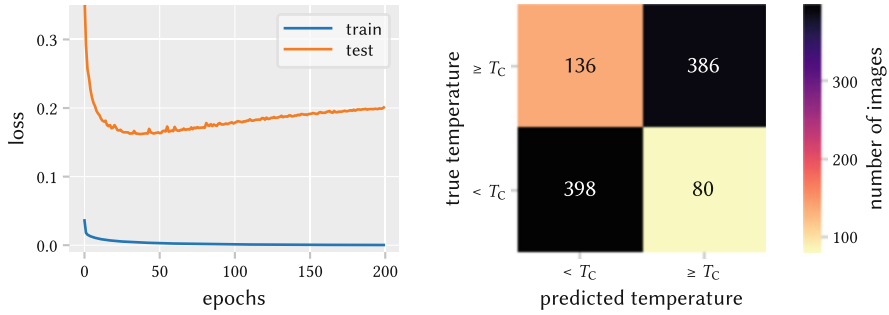


Fig. 18.9 Classification of the Ising microstructures w.r.t. to temperature. The left panel shows the training and testing errors, the right panel shows the confusion matrix

From the confusion matrix, we can see that the model has more difficulties to predict microstructures that were obtained at higher temperatures: 136 examples out of 522 examples for microstructures above the Curie temperature were wrongly classified. Note that due to the random split into training and testing data, there is a small class imbalance, which is why we should not directly compare the number of misclassified images and instead should use relative numbers: for the “high temperature” data, 25% of the images were misclassified, while for the “low temperature” data only 15.6% were misclassified.

To analyze if there specific temperature values for which microstructures are more prone to misclassifications than others, we could turn the two-class classification problem into a multi-class classification problem, e.g., using 16 classes of temperature ranges. Obviously, one could even increase the number of classes such that in the limit case, we use the classifier for solving a regression problem. Alternatively, we can also directly turn this into a regression problem. The network architecture requires then that the output layer contains only a single unit. Additionally, the labels of the data records are numeric labels containing the temperature. Figure 18.10 shows the results after training.

Ideally, all data points should be located on the dark, solid line. To quantify how well the data fits to this line, we calculate the *coefficient of determination* R^2 and the *correlation coefficient* corr , compare Chap. 9. We find $R^2 \approx 0.28$ where a perfect fit would be denoted by 1.0. Furthermore, it is $\text{corr} \approx 0.64$ which according to Sect. 8.7.3 denotes a positive correlation that is in between moderate and strong. It is interesting to observe that the microstructure examples that correspond to the Curie temperature are those that can be easiest predicted. Both implementations show good results from which it is obvious that the network learned something about the microstructures. Using more advanced network architectures, some of which will be introduced in the next chapter, however, can drastically increase the predictive capabilities. A systematic study where the performance of a range of different models is investigated can be found in NGUYEN et al. [5].

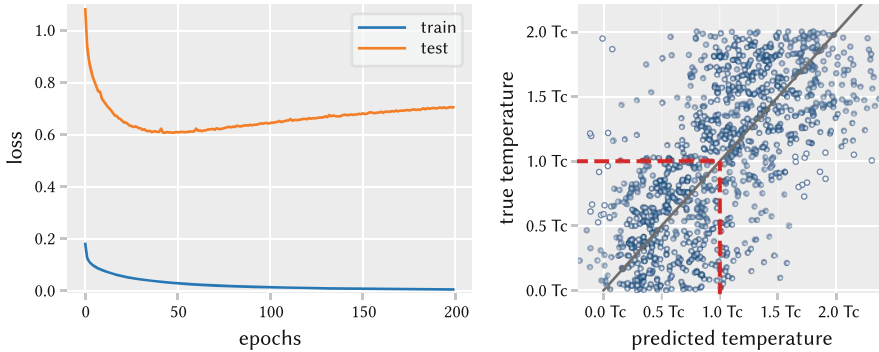


Fig. 18.10 The Ising dataset as a regression problem. The left panel shows the training and testing errors, the right panel shows the predicted vs the true temperature for the testing dataset

18.9 Exercises

18.1. In Sect. 18.3.3, we implemented a first model that performs backpropagation. In this exercise, you should implement the tracking of the mean testing loss and reproduce the evolution of the train/test losses from Fig. 18.3. Hint: in the `for` loop over all epochs, use the current weights and bias values to predict the labels for *training* data. Use the previously introduced function for splitting the dataset into training and testing data.

18.2. The derivation and Python implementation from Sect. 18.3.3 uses three biases.

- Is the third bias really needed or can this be “taken over” by the hidden units? Do an experiment with the code and discuss the results.
- How important is the type of initialization of the weights and biases? Perform an experiment with the Python code.

18.3. Use the code of the fully connected network to create a network architecture with which the XOR problem can be solved. The XOR function takes two input values, each of which is either 0 or 1. If both inputs have the same value, then the XOR function yields zero; if the inputs have different values (0 and 1, or 1 and 0), the output is 1.

Which is a “minimal network architecture”, e.g., in terms of number of layers and number of nodes, that still gives the desired result?

18.4. Compare the sigmoidal activation functions with the tanh and the ReLU activation function. Use the code for a fully connected network from the companion website, pick a dataset of your choice, and compare their performance. Also, measure the compute time required for evaluating the activation functions for a

single number and for an array of 100,000 elements. Which activation function is preferable?

18.5. Extend the code for a fully connected network used in Sect. 18.5.2 and perform some experiments.

- (a) Extend the code such that during the training also the test cost (MSE) is computed and recorded.

Hint: In the method `train` of the class `Network` add your code after back-propagation of the errors. The training cost is then computed with the *current* values of the weights and biases, based on the *testing* dataset.

- (b) What do you observe for the training and testing MSE when you increase the number of epochs or/and the learning rate? Discuss the extreme parameter combinations.
- (c) Can you manage to bring the model to a point where it overfits?

18.6. Use the code of the fully connected network from Sect. 18.5.2 to investigate the influence of different activation functions. In particular, compare the performance and discuss the behavior of the (i) sigmoidal, (ii) tanh, (iii) ReLU, and (iv) a leaky ReLU function. Which one is the best performing function for the investigated problem? For (iv) try different leakage values.

18.7. Extend the code for a fully connected network used in Sect. 18.5.2 such that the training is done using mini-batches. Compare this implementation with the one without mini-batches.

- (a) How do they perform in terms of stability or efficiency?
- (b) What is a good batch-size and what is the difference between a too large or too small mini-batch size?

18.8. Use the code from Sect. 18.5.2 and the extended version from Exercise 18.6 and answer the following questions:

- (a) What is the influence of the learning rate on the performances of the network concerning the training/testing data?
- (b) What is the impact of the number of epochs on the performances? (Hint: Investigate the training in terms of overfitting)
- (c) Does increasing the size of the network (i.e., the number of layers and the number of nodes per layers) lead to better performances? Discuss it with an example

18.9. In Sect. 18.6.3, we saw how weights can or should be initialized. Try different initialization methods (uniform, normal, constant, Xavier, He) and discuss the results and performances.

18.10. We saw that a neural network can solve the XOR problem. Can a neural network solve any combination of logical gate? Try this out for a number of combinations of different logical (Boolean) elements. What would be the condition to ensure that there exists a solution?

18.11. Investigate the “most confused” images further. Identify from the confusion matrix those images that get most often confused with a different digit. Locate these images and try to understand why they are misclassified the way how they are.

Bonus task: perform a principal component analysis on the same dataset as used here and interpret your findings in view of these results.

18.12. Use the code of the fully connected network from Sect. 18.5.2, and train it on the dataset DS-2-small (UCI Handwritten Digits). How do the predictions change if a new image is rotated by multiples of 90° ? Translational invariance is more difficult to be investigated since the image are very small and most of the digits fill the whole image. You can still investigate this for one of the more narrow digits, e.g., for a “1”. What do you observe?

References

1. X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh and M. Titterton, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, Chia Laguna Resort, Sardinia, Italy, 13–15 May 2010. PMLR. URL <https://proceedings.mlr.press/v9/glorot10a.html>.
2. K. He, X. Zhang, S. Ren and J. Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1026–1034, 2015. DOI <https://doi.org/10.1109/ICCV.2015.123>.
3. C. Hill. *Learning Scientific Programming with Python*. Cambridge University Press, 2nd edition. ISBN 978-1108745918.
4. S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.
5. B. D. Nguyen, P. Potapenko, A. Dermici, K. Govind, and S. Sandfeld. Forward modeling of materials science simulations: learning the microstructure-property relation. *under review*.
6. F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. DOI <https://doi.org/10.1037/h0042519>.
7. F. Rosenblatt. Principles of neurodynamics: Perceptrons and the theory of brain mechanisms, 1962.
8. D. Rumelhart, G. Hinton, and R. Williams. Learning representations by back-propagating errors. *Nature* 323:533–536, 1986. DOI <https://doi.org/10.1038/323533a0>
9. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL <http://jmlr.org/papers/v15/srivastava14a.html>.
10. P. J. Werbos. Beyond regression—new tools for prediction and analysis in the behavioral sciences, 1974.
11. P. J. Werbos. *The Roots of Backpropagation: From Ordered Derivatives to Neural Networks and Political Forecasting*. Wiley-Interscience, USA, 1994. ISBN 0471598976.
12. B. Yuen, M. T. Hoang, X. Dong, and T. Lu. Universal activation function for machine learning. *Scientific Reports*, 11(1), Sept. 2021. DOI <https://doi.org/10.1038/s41598-021-96723-8>.